Лабораторна робота №6

Тема: Конвертація даних між форматами CSV та JSON.

Мета: Освоїти роботу з файлами даних у форматі csv та json

КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Формат CSV.

Робота з файлами CSV (**Comma-Separated Values**) є базовою навичкою для будь-якого розробника, оскільки цей формат є стандартом де-факто для обміну табличними даними між різними системами (Excel, бази даних, вебдодатки).

1. Структура формату

CSV − це текстовий формат, де кожен рядок файлу відповідає одному рядку таблиці, а колонки розділені спеціальним символом-розділювачем.

Розділювач (Delimiter). Найчастіше це кома (,), але в локалізованих системах (наприклад, українських) часто використовується крапка з комою (;), оскільки кома слугує десятковим розділювачем у числах.

Рядок заголовків (Header). Зазвичай перший рядок містить назви колонок, хоча це не є обов'язковим.

Текстові обмежувачі (Quotes). Якщо текст містить розділювач або перенос рядка, його обов'язково беруть у подвійні лапки "".

2. Основні правила (Специфікація RFC 4180)

Хоча суворого стандарту тривалий час не існувало, більшість систем дотримуються правил RFC 4180:

- Кожен запис розташовується на окремому рядку.
- Останній запис у файлі може не мати символу перенесення рядка.
- Якщо всередині поля є лапки, вони мають бути екрановані ще одними лапками: ""Привіт"""".
- Типи даних не визначаються: все вважається текстом, а програма-інтерпретатор сама вирішує, де число, а де дата.

3. Програмна обробка

При роботі з CSV важливо не "винаходити велосипед", намагаючись розпарсити файл просто через `split(',')`, оскільки це зламає логіку при наявності ком усередині тексту.

Для програмної обробки використовують використовуються вбудовані функції:

- `fgetcsv()` для читання. Отримує рядок із покажчика файлу та аналізує поля CSV.
- `fputcsv()` для запису. Форматування рядка у вигляді CSV та його запису у відкритий файл

Вони автоматично обробляють екранування та лапки.

Приклад: Вивести на екран вміст файлу `Edu_Comp_agro.csv`. У першому рядку назви полів.

Фрагмент файлу:

"id","name_file","name","status","id_dep"

"12","w208mF-R01-20","Економіка технологічних систем","1","31"

Код парсингу CSV файлу

```
<?php
$filename = 'Edu_Comp_agro.csv';
if (file_exists($filename)) {
    echo "Файл знайдено <br>";

    // Відкриваємо файл для читання
    if (($f = fopen($filename, 'r')) !== FALSE) {

        // Читаємо рядок за рядком
        while (($row = fgetcsv($f, 1000, ",")) !== FALSE)
{

            // Перевіряємо, чи рядок не порожній
            if ($row !== null) {
                foreach ($row as $value) {
                    echo htmlspecialchars($value) . ' ';
                }
                echo "<br>";
            }
        }
        fclose($f);
    }
} else {
    echo "Файл відсутній";
}
?>
```

Формат JSON

JSON (**JavaScript Object Notation**) — це легкий формат обміну даними, який став стандартом у вебдевлопменті завдяки своїй простоті, легкості читання як людьми, так і машинами, а також повній незалежності від мов програмування.

1. **Формат структури**

Дані в JSON базуються на двох структурах:

- **Об'єкт:** Набір пар «ключ: значення», взятий у фігурні дужки `{ }`. Ключ завжди є рядком у подвійних лапках.
- **Масив:** Впорядкований список значень, взятий у квадратні дужки `[ ]`.

Допустимі типи даних:

1. **Рядок** (String) − обов'язково у подвійних лапках: `"name": "Toyota"`.

2. **Число** (Number) – ціле або з рухомою комою: `"price": 25000.50`.
3. **Логічне значення** (Boolean) – `true` або `false`.
4. **Масив** (Array) – список значень будь-якого типу: `[1, 2, 3]`.
5. **Об'єкт** (Object) – вкладені структури: `{"details": {"color": "red"}}`.
6. **Null** – порожнє значення: `"description": null`.

2. Робота з JSON у PHP

Для роботи з цим форматом у PHP передбачено дві основні функції:

`json_encode()` – створення JSON

Перетворює масив або об'єкт PHP у JSON-рядок. Це необхідно, наприклад, для передачі даних із сервера на фронтенд.

```
$data = ["status" => "success", "code" => 200];
echo json_encode($data); // Результат: {"status":"success","code":200}
```

`json_decode()` – читання JSON

Перетворює JSON-рядок у формат, з яким може працювати PHP.

- `json_decode($json)` — повертає об'єкт (`stdClass`).
- `json_decode($json, true)` — повертає асоціативний масив (найчастіше використовується в навчальних завданнях).

3. Програмна обробка.

Функція `file_get_contents()` у PHP є найбільш простим та ефективним способом зчитати вміст файлу в один рядок. Вона ідеально підходить для роботи з JSON-файлами, конфігураціями або отримання HTML-коду сторінок через URL.

Основне призначення

На відміну від комбінації fopen(), fread() та fclose(), ця функція виконує всі дії за один крок: вона відкриває файл, зчитує весь його вміст у пам'ять і автоматично закриває з'єднання.

Синтаксис

`file_get_contents(string $filename, bool $use_include path = false, resource $context = null, int $offset = 0, int $length = null)`

- `$filename` – шлях до файлу або URL-адреса.
- `$offset` – позиція, з якої починати зчитування.
- `$length` – максимальний розмір даних, які потрібно зчитати.

Приклад

```
$json_data = file_get_contents('data.json');
$array = json_decode($json_data, true);
```

Функція **`file_put_contents()`**– призначена для швидкого запису даних у файл. Це значно зручніше, ніж послідовне використання `fopen()`, `fwrite()` та `fclose()`.

1. Основне призначення.

Функція бере рядок (або масив) і записує його у вказаний файл. Якщо файлу не існує, PHP спробує його створити. Якщо файл уже існує, за замовчуванням його вміст буде повністю замінено на новий.

2. Синтаксис.

`file_put_contents(string $filename, mixed $data, int $flags = 0, resource $context = null)`

- `$filename` – шлях до файлу, куди пишемо дані.
- `$data` – рядок, масив або потік даних. Якщо передати масив, він буде автоматично "склеєний" (аналог `implode('', $array)`).
- `$flags` – Спеціальні параметри запису (найважливіша частина).

3. Ключові прапорці (Flags)

Саме прапорці роблять цю функцію гнучкою:

- **`FILE_APPEND`**: Якщо цей прапорець встановлено, дані не перезаписують файл, а додаються в його кінець (корисно для логів).
- **`LOCK_EX`**: Встановлює ексклюзивне блокування на файл під час запису. Це заважає іншим процесам писати в цей же файл одночасно, запобігаючи пошкодженню даних.

Приклад

```
$user_data = [
    "name" => "Олексій",
    "role" => "admin"
];

// Перетворюємо масив у JSON і записуємо
file_put_contents('user.json',  json_encode($user_data,
JSON_UNESCAPED_UNICODE));
```

ЗАВДАННЯ ДЛЯ САМОСТІЙНОГО ВИКОНАННЯ.

***Завдання 1.* Парсер файлу CSV.**

Сценарій. Дано файл `cars.csv` у якому міститься про авто які продаються в автосалоні. Ознайомитися з структурою даних. Спробуйте визначити призначення полів. Перший рядок містить назву полів.

Завдання. Скласти php сценарій для виводу наступної інформації:

1. Вивести інформацію з файлу у вигляді таблиці.

2. Вивести інформацію з файлу у вигляді таблиці про авто марки "`Ford`".

3. Вивести інформацію з файлу у вигляді таблиці про японського виробника.

4. Вивести інформацію про ціну авто “`Min.Price`”, “`Price`”, “`Max.Price`”.

5. Вивести марки авто у яких `Cylinders>=6`

6. Вивести повноприводні авто "`DriveTrain`".

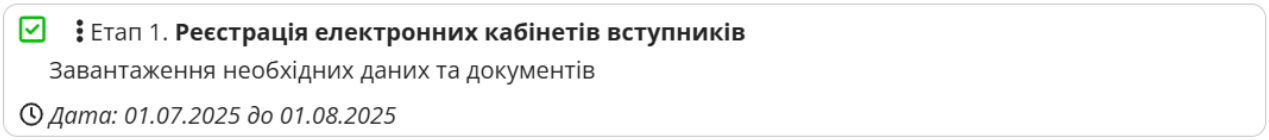
7. Визначити сумарну вартість авто по салону “`Price`”

Завдання 2. **Дорожня карта вступника.**

Сценарій. При вступі до закладу вищої освіти необхідно дотримуватися дорожньої карти вступу. Вступ розбитий на етапи, які мають свої часові рамки виконання. Саме ці етапи формують дорожню карту вступу. На виконання кожного етапу вступник виконує певні дії.

Завдання. На основі файлу `road_bachelor.json`, що містить масив об'єктів (етапів вступу). Кожен етап має: назву, опис, дату початку та дату завершення. Створити дорожню карту вступу на ОС «Бакалавр» у форматі логічної послідовності блоків.

Приклад формату блоку.



Завдання 3. **Перетворення форматів.**

- Перетворити дані файлу `cars.csv` (завдання 1) у файл даних формату JOSN – `cars.json`
- Перетворити дані файлу `road_bachelor.json` (завдання 2) у файл даних формату CSV – `road_bachelor.csv`